

# Reviewer Pack — Restriction-Local Fourier Doubling Gap for ACC[2] vs Sipser

Entry 51d7c2e9512a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 18:51:57 UTC

## Restriction-Local Fourier Doubling Gap for ACC[2] vs Sipser

**Entry ID:** 51d7c2e9512a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 18:51:57 UTC

### 1. Conjecture

**Field A** (mathematical branch): Additive combinatorics: Plünnecke-Ruzsa doubling of heavy Fourier supports in  $F_2^k$  **Field B** (complexity object): ACC[2] circuits and the depth-d Sipser function (worst-case lower bound via average over restrictions)

#### Statement:

For  $f: \{0,1\}^n \rightarrow \{\pm 1\}$  and a random restriction  $\rho$  that zeros a uniformly random  $(n-k)$ -subset of inputs and keeps  $k = \lceil \log_2 n \rceil$  alive, let  $f|_\rho$  be the resulting  $k$ -variable function with Walsh-Hadamard spectrum  $\hat{g}$ . Define  $HF(g) = \{S \subseteq [k] : |\hat{g}(S)| \geq 1/4\}$  and  $\text{Doub}(f) = E_\rho[|HF(f|_\rho)| / \max(1, |HF(f|_\rho)|)]$ . Conjecture: (i) every  $AC^0[\oplus]$  (a.k.a. ACC[2]) circuit  $C$  of size  $s$  and depth  $d$  satisfies  $\text{Doub}(f_C) \leq (1 + \log s)^{2d}$ ; (ii) the depth-3 Sipser function  $S_n$  with alternating fan-ins  $\approx n^{1/3}$  satisfies  $\text{Doub}(S_n) \geq \sqrt{(\log n)/6}$  for all  $n \geq 16$ . A single  $(n, \text{seed})$  instance violating either bound refutes the conjecture.

#### Rationale (proposer's reasoning):

Razborov-Smolensky says ACC[2] functions are approximated by low-degree  $F_2$ -polynomials, whose Fourier mass is concentrated on a low-weight, additively-structured (Freiman-like) family of characters; restricting to a log-sized window preserves this structure, so the heavy support inherits small doubling. Sipser's recursive AND-OR-AND amplifier instead spreads heavy Fourier mass across characters with no  $F_2$ -affine structure, so restriction averaging exposes a doubling gap. The construction escapes natural proofs by being non-large: random  $f$  has empty HF after restriction (so  $\text{Doub}=0$ ), meaning random functions trivially satisfy the ACC[2] bound — the property does not distinguish random from easy, only Sipser from easy.

**Taxonomy category:** AVG\_T0\_WORST\_CASE (status at proposal time: partially\_alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** a4df5696d5f5589f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion:**

Across  $n \in \{16, 24, 32, 40\}$  with 30 seeds each (120 instances), the conjecture is SUPPORTED iff for every  $n$  the median  $\text{Doub}(\text{ACC}[2]) \leq (1 + \log_2 s)^6$  AND median  $\text{Doub}(\text{Sipser}_n) \geq \sqrt{(\log_2 n)/6}$ , with zero single-seed violations of either bound. Any one  $(n, \text{seed})$  breaking either inequality FALSIFIES.

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.92       | SAFE             | SAFE             |
| NATURAL_PROOFS | SAFE    | 0.90       | SAFE             | SAFE             |
| ALGEBRIZATION  | SAFE    | 0.90       | SAFE             | SAFE             |
| KARP_LIPTON    | SAFE    | 0.95       | SAFE             | SAFE             |

### 4. Novelty audit

**Verdict:** NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - Plünnecke-Ruzsa Fourier doubling AC0 parity lower bound - random restriction Walsh-Hadamard heavy spectrum Sipser ACC - additive combinatorics sumset Fourier support circuit complexity AC0[2]

**Top relevant hits considered:** - [<http://arxiv.org/abs/2511.07739v3>] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [<http://arxiv.org/abs/1712.07615v2>] A Plünnecke-Ruzsa inequality in compact abelian groups - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/1508.06336v1>] SPRIGHT: A Fast and Robust Framework for Sparse Walsh-Hadamard Transform - [<http://arxiv.org/abs/nuclex/0104014v1>] Heavy-Ion Physics at the LHC - [<http://arxiv.org/abs/1912.00015v1>] Efficient Approximate Inference with Walsh-Hadamard Variational Inference - [<http://arxiv.org/abs/1904.05483v2>] Parallels Between Phase Transitions and Circuit Complexity? - [<http://arxiv.org/abs/math/0409420v1>] Finite field models in additive combinatorics

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def evaluate_circuit(circuit, input):
        stack = []
        for gate in circuit:
            if gate == 'AND':
                a = stack.pop()
                b = stack.pop()
                stack.append(a and b)
            elif gate == 'OR':
```

```

        a = stack.pop()
        b = stack.pop()
        stack.append(a or b)
    elif gate == '⊕':
        a = stack.pop()
        b = stack.pop()
        stack.append(a != b)
    else:
        stack.append(gate(input))
return stack[0]

def walsh_hadamard_transform(f, n):
    N = 2**n
    result = [0] * N
    for i in range(N):
        for j in range(N):
            result[i] += f(j) * (-1)**(i & j)
    return [x / math.sqrt(N) for x in result]

def hamming_weight(x):
    return bin(x).count('1')

def freiman_family_size(k, n):
    return 2**(n - k + 1)

def doubling(g, tau=0.25):
    N = len(g)
    HF = [g[i] for i in range(N) if abs(g[i]) >= tau]
    return sum(abs(HF[i] - HF[j]) for i in range(len(HF)) for j in range(i+1, len(HF))) / max(1, len(HF))

def generate_random_circuit(n):
    circuit = []
    for _ in range(n):
        gate = random.choice(['AND', 'OR', '⊕'])
        if gate == '⊕':
            circuit.append(lambda x: x % 2)
        else:
            circuit.append(random.randint(0, n-1))
    return circuit

def sipser_function(n, i):
    return (i & (i + 1)) != 0

n_values = [16, 24, 32, 40]
k_values = [math.ceil(math.log2(n)) for n in n_values]

total_doubling_acc = []
total_doubling_sipser = []

for n, k in zip(n_values, k_values):
    s = math.floor(n**1.5)
    circuit = generate_random_circuit(s)

    for _ in range(30):
        k_window = random.sample(range(n), k)
        f_acc = [evaluate_circuit(circuit, i) for i in k_window]

```

```

g_acc = walsh_hadamard_transform(f_acc, k)
total_doubling_acc.append(doubling(g_acc))

f_sipser = [sipser_function(n, i) for i in range(2**n)]
g_sipser = walsh_hadamard_transform(f_sipser, n)
total_doubling_sipser.append(doubling(g_sipser))

median_doubling_acc = sorted(total_doubling_acc)[len(total_doubling_acc) // 2]
median_doubling_sipser = sorted(total_doubling_sipser)[len(total_doubling_sipser) // 2]

conjecture_holds = (median_doubling_acc <= (1 + math.log2(s))**6 and
                    median_doubling_sipser >= math.sqrt(math.log2(n))/6)

return {
    "metric_name": "Doubling Gap",
    "metric_value": median_doubling_acc,
    "instances_tested": len(total_doubling_acc),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"ACC: {median_doubling_acc}, Sipser: {median_doubling_sipser}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_doubling_acc = [r["metric_value"] for r in results if "ACC" in r["counterexample"]]
    total_doubling_sipser = [r["metric_value"] for r in results if "Sipser" in r["counterexample"]]

    mean_doubling_acc = sum(total_doubling_acc) / len(total_doubling_acc)
    std_doubling_acc = math.sqrt(sum((x - mean_doubling_acc)**2 for x in total_doubling_acc) / len(total_doubling_acc))
    support_fraction_acc = sum(1 for r in results if "ACC" not in r["counterexample"]) / len(results)

    mean_doubling_sipser = sum(total_doubling_sipser) / len(total_doubling_sipser)
    std_doubling_sipser = math.sqrt(sum((x - mean_doubling_sipser)**2 for x in total_doubling_sipser) / len(total_doubling_sipser))
    support_fraction_sipser = sum(1 for r in results if "Sipser" not in r["counterexample"]) / len(results)

    if all("ACC" not in r["counterexample"] and "Sipser" not in r["counterexample"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_doubling_acc} std={std_doubling_acc} support_fraction={support_fraction_acc}")
    elif any("ACC" in r["counterexample"] for r in results) and all("Sipser" not in r["counterexample"] for r in results):
        print(f"RESULT: FALSIFIED counterexample=\"ACC\" first_failing_seed={seeds[next(i for i, r in enumerate(results) if 'ACC' in r['counterexample'])]}")
    elif any("Sipser" in r["counterexample"] for r in results) and all("ACC" not in r["counterexample"] for r in results):
        print(f"RESULT: FALSIFIED counterexample=\"Sipser\" first_failing_seed={seeds[next(i for i, r in enumerate(results) if 'Sipser' in r['counterexample'])]}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_246f894e.py", line 112, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_246f894e.py", line 84, in run_trial
    f_acc = [evaluate_circuit(circuit, i) for i in k_window]
             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_246f894e.py", line 37, in evaluate_circ
    stack.append(gate(input))
                  ^^^^^^^^^^^
TypeError: 'int' object is not callable
```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed with a `TypeError` ('int' object is not callable) in `evaluate_circuit` before producing any data, so neither the support nor falsification condition can be evaluated. | next: Fix the circuit representation/evaluation bug (gate appears to be stored as an int rather than a callable) and re-run the 120-instance protocol.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | claude_max    | opus             | 0         | 0   | 156112       |
| 2 | preregistration | claude_max    | opus             | 0         | 0   | 7484         |
| 3 | novelty         | claude_max    | opus             | 0         | 0   | 3549         |
| 4 | novelty         | claude_max    | opus             | 0         | 0   | 11878        |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15072        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 14413        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 16516        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17225        |
| 9 | judge           | claude_max    | opus             | 0         | 0   | 4823         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 247072 ms total latency. Provider mix: {'claude\_max': 5, 'ollama\_remote': 4}

(full prompt+response transcripts available in `research/audit/51d7c2e9512a.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/51d7c2e9512a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/51d7c2e9512a.tar.gz` (if generated)